

实验二：C- 语义分析实验报告

1 程序功能与实现方法

本程序在实验一词法/语法分析器的基础上，手写实现了 C- 语言的语义分析器，支持必做 17 种语义错误检查及选做要求 3.3（结构等价）。

1.1 总体架构

语义分析采用**两遍遍历**策略，基于实验一构造的 AST 进行：

第一遍（声明收集）：遍历 AST，将结构体定义、函数签名（含返回类型与形参类型）、全局变量声明插入符号表。此遍确保后续表达式检查时所有声明已就位，支持前向引用。

第二遍（语句检查）：再次遍历 AST，检查每个函数体中的局部变量声明与所有语句、表达式。对每个表达式推导其类型并验证类型约束，对每个语句检查控制流条件、返回类型等。

两遍共用同一个符号表，通过全局变量 `last_err_line` 与实验一的词法/语法错误报告协调，确保同行不重复报错。

1.2 类型系统（type.h/c）

类型采用书中推荐的链表结构表示：

- **基本类型**：TYPE_INT、TYPE_FLOAT
- **数组类型**：{Type* elem; int size;} 链表嵌套支持多维数组
- **结构体类型**：{char* name; FieldList* fields;} 域链表存储域名与类型
- **函数类型**：{Type* ret; int pc; Type** params;}

`type_equal` 递归比较两类型是否等价。对结构体类型，逐域比较域名与类型（结构等价），而非仅比较结构体名（名等价），从而实现选做要求 3.3。

1.3 符号表（symbol.h/c）

采用书中推荐的 **PJW 哈希函数**配合**链地址法**实现散列表，桶数 16384。每个符号记录名称、种类（变量/函数/结构体）、类型指针、定义行号和哈希链指针。由于实验假设所有变量为全局作用域，采用单层符号表即满足需求。

1.4 语义检查实现

17 种错误类型的检查分布在 AST 遍历的各个阶段：

- **声明阶段**：检查错误类型 3（变量重定义）、4（函数重定义）、15（结构体域重定义/初始化）、16（结构体名重复）。函数形参在声明阶段即作为全局变量插入符号表。

- **表达式阶段** (`type_of_exp`): 递归推导表达式类型的同时检查错误 1 (变量未定义)、2 (函数未定义)、5 (赋值类型不匹配)、6 (左值检查)、7 (操作数类型不匹配)、9 (函数调用实参不匹配)、10–14 (数组/结构体/域相关错误)。
- **语句阶段** (`check_statement`): 检查错误 8 (返回类型不匹配) 以及 if/while 条件必须为 int 型。
- **类型引用阶段** (`specifier_to_type`): 当变量声明引用未定义结构体时, 检查错误 17。

表达式类型推导采用根据 AST 子节点模式匹配的方法: 对于二目运算符, 分别计算左右操作数类型后检查兼容性; 对于函数调用, 查符号表获取形参列表后逐一比对实参; 对于数组和结构体访问, 先在符号表中解析基类型再递归推导。

2 编译方法

在含 Makefile 的 Code 目录下执行 `make` 即可编译。依赖: `gcc, flex, bison, -lfl, -ly`。编译后生成 `parser`, 运行方式为 `./parser < 输入文件 >`。

相较于实验一, 本次新增 6 个源文件 (`type.c, symbol.c, semantic.c` 及对应头文件), Makefile 通过 `find` 自动发现所有 `.c` 文件, 无需手动修改。

3 设计亮点

1. **结构等价实现** (要求 3.3)。 `type_equal` 递归比较结构体的每个域的类型, 域类型若为嵌套结构体或数组则继续递归。这使得 `struct{int a; float b;}` 与 `struct{int x; float y;}` 被视为等价类型, 允许相互赋值。
2. **表达式类型推导的递归设计**。 `type_of_exp` 是语义分析的核心函数, 通过检查 AST 节点的子节点模式来识别表达式类别 (变量引用、字面量、函数调用、各类双目/单目运算、数组/结构体访问), 递归推导子表达式类型后合成当前表达式类型, 在此过程中集中完成 10 余种类型错误的检查。这种设计将类型检查逻辑收敛到单一函数, 避免了分散检查带来的遗漏风险。
3. **跨阶段错误去重**。通过 `last_err_line` 全局变量在词法分析器 (实验一)、语法分析器 (实验一) 和语义分析器 (实验二) 之间协调: 每个模块在报告错误前检查该变量, 确保同一行的多个潜在错误只报告第一个。 `SEM_ERROR` 宏使用 `do{...}while(0)` 包装, 避免宏展开导致的悬挂 else 问题。
4. **DefList 递归遍历**。C- 文法的 DefList 是递归结构 (`Def DefList | ε`), 而非平坦列表。编写 `walk_deflist` 辅助函数正确处理递归结构, 避免遗漏嵌套层中的定义。